

# 1. Conjunto independiente máximo en intervalos

**Problema.** Dados  $n$  intervalos  $a_1 \dots a_n$  con  $a_i = [s_i, t_i)$  queremos encontrar un conjunto de máxima cardinalidad de intervalos que no se superpongan<sup>1</sup>. Suponemos para simplificar el problema que los momentos iniciales y finales son todos distintos.

**Algoritmo.** Primero ordenamos los intervalos por momento final de menor a mayor, e inicializamos una solución parcial  $G$  como vacía. Luego, recorremos los intervalos de izquierda a derecha y en cada iteración  $i$  vemos si el intervalo  $i$  tiene intersección vacía con los intervalos de  $G$ . Si eso pasa, lo agregamos a  $G$ . Caso contrario, lo ignoramos y seguimos al siguiente intervalo.

Más formalmente: mantenemos una solución parcial  $G$  que empieza vacía, y recorremos  $a_1, \dots, a_n$ . Agregamos  $a_i$  a  $G$  si  $G = \emptyset$  o si  $s_i \geq t_\ell$ , donde  $a_\ell$  es el último intervalo agregado a  $G$ .

Para probar que este algoritmo es correcto (o sea, que devuelve un subconjunto independiente máximo) vamos a dar un invariante para el ciclo. Sea  $G_i$  la solución parcial al terminar la iteración  $i$  y  $I_i = \{a_1, \dots, a_i\}$  (por lo que  $G_0 = I_0 = \emptyset$ ). El invariante que vamos a probar es:

$$P(i) : \text{ existe una solución óptima } O \text{ tal que } O \cap I_i = G_i.$$

Es decir, al finalizar la iteración  $i$  la solución parcial  $G_i$  coincide en las decisiones tomadas con alguna solución óptima hasta la actividad  $i$ . Probemos entonces que el ciclo que describimos satisface este invariante.

**Inicialización.**  $P(0)$  vale: cualquier óptima  $O$  satisface  $O \cap I_0 = \emptyset = G_0$ .

**Mantenimiento.** Supongamos que vale  $P(i)$  y tomemos una solución óptima  $O$  con  $O \cap I_i = G_i$ . Consideramos el intervalo  $a_{i+1}$ .

- Si el algoritmo descarta  $a_{i+1}$  entonces este intervalo se superpone con el último agregado a  $G_i$ . Como  $G_i \subseteq O$  y  $O$  es independiente,  $a_{i+1} \notin O$ . Por lo tanto,  $O \cap I_{i+1} = G_i = G_{i+1}$ .
- Si el algoritmo agrega  $a_{i+1}$  tenemos  $G_{i+1} = G_i \cup \{a_{i+1}\}$ . Si  $a_{i+1} \in O$ , la misma solución óptima sirve para concluir  $P(i+1)$ .  
Si  $a_{i+1} \notin O$ , consideremos el intervalo  $a_j \in O \setminus G_i$  que termine lo antes posible (notar que, como por el invariante  $G_i$  y  $O$  coinciden en todos los intervalos de  $I_i$ , debe valer que  $j > i+1$ ). Como los intervalos están ordenados por finalización, tenemos que  $t_{i+1} < t_j$ . Definimos

$$O' = (O \setminus \{a_j\}) \cup \{a_{i+1}\}.$$

Notemos que este conjunto es independiente:  $a_{i+1}$  es compatible con  $G_i$  por la condición del algoritmo. Por otro lado, todo intervalo de  $O \setminus (G_i \cup \{a_j\})$  empieza luego de  $t_j$ , pues  $a_j$  es el primer intervalo en terminar entre los intervalos restantes de  $O$ . Es decir, si  $a_\ell \in O \setminus (G_i \cup \{a_j\})$ , entonces  $t_j < s_\ell$ . Como  $t_{i+1} < t_j$ , estos tampoco se superponen con  $a_{i+1}$ , pues podemos concluir que  $t_{i+1} < t_j < s_\ell$ . Además,  $|O'| = |O|$ , por lo que concluimos que  $O'$  es otra solución óptima. Resulta entonces que  $O' \cap I_{i+1} = G_i \cup \{a_{i+1}\} = G_{i+1}$ , como queríamos probar.

Con esto concluimos que vale el invariante. Notemos que al final, cuando vale  $P(n)$ , tenemos que  $O = O \cap I_n = G_n$  para alguna solución óptima  $O$ . Y esto implica precisamente que la solución parcial que construimos coincide exactamente con una solución óptima.

<sup>1</sup>Dos intervalos  $[s, t)$  y  $[s', t')$  se superponen si  $s \leq s' \leq t$  o si  $s' \leq s \leq t'$

## 2. Algoritmo de Prim

**Problema.** Dado un grafo conexo  $G = (V, E)$  con pesos  $w : E \rightarrow \mathbb{R}$ , queremos encontrar un árbol generador mínimo (AGM) de  $G$ .

**Algoritmo.** Partimos de un vértice  $r$  arbitrario y empezamos con el árbol  $T_0$  formado por  $r$  y ninguna arista. En cada iteración tomamos una arista de peso mínimo en la *frontera* del árbol actual, es decir, entre las aristas con exactamente un extremo en  $T_i$ , y la agregamos a  $T_i$  para formar  $T_{i+1}$ . Repetimos este proceso  $n - 1$  veces, lo que termina construyendo un árbol.

Para ver la correctitud de este algoritmo, vamos a dar un invariante y probar su validez de forma similar a como hicimos antes. Si  $T_i$  es la solución parcial luego de  $i$  iteraciones, vamos a probar que vale

$$P(i) : \quad T_i \text{ tiene } i \text{ aristas, es un árbol y está contenido en algún AGM.}$$

**Inicialización.**  $T_0$  tiene 0 aristas y es un árbol de un solo vértice. Además, está contenido en cualquier AGM. Por lo tanto, vale  $P(0)$ .

**Mantenimiento.** Supongamos  $P(i)$  y sea  $A$  un AGM que contiene a  $T_i$ . Escribimos  $S_i = V_{T_i}$ . Si  $S_i \neq V$ , por conexidad existe alguna arista en la frontera. Sea  $vw$  la elegida por el algoritmo, con  $v \in S_i$  y  $w \notin S_i$ , de modo que

$$T_{i+1} = T_i + vw.$$

$T_{i+1}$  es un árbol: agregar una arista hacia un vértice que antes no estaba en  $T_i$  no puede crear un ciclo. Luego, solo falta ver  $T_{i+1}$  está contenido en un AGM. Vemos esto por casos.

- Si  $vw \in E(A)$ , entonces  $T_{i+1} \subseteq A$  y terminamos.
- Si  $vw \notin E(A)$ , consideramos el único camino de  $v$  a  $w$  en  $A$ . Como empieza en  $S_i$  y termina fuera de  $S_i$ , alguna de sus aristas cruza la frontera. Llamamos  $xy$  a una de ellas, con  $x \in S_i$  e  $y \notin S_i$ . Como  $vw$  tiene peso mínimo en la frontera,

$$c(vw) \leq c(xy).$$

Al agregar  $vw$  a  $A$  se forma un único ciclo: el camino de  $v$  a  $w$  en  $A$  junto con la nueva arista. Como  $xy$  está en ese ciclo, al quitarla obtenemos otro árbol generador

$$A' = A + vw - xy,$$

cuyo peso satisface

$$c(A') = c(A) + c(vw) - c(xy) \leq c(A).$$

Como  $A$  ya era mínimo,  $A'$  también es un AGM. Claramente tenemos  $T_{i+1} \subseteq A'$ , por lo que probamos  $P(i + 1)$ .

Cuando el invariante vale en el paso final concluimos que, gracias a  $P(n - 1)$ ,  $T_{n-1}$  es un árbol con  $n - 1$  aristas contenido en un AGM  $A$ . Como  $A$  también tiene  $n - 1$  aristas, ambos árboles coinciden, y entonces  $T_{n-1}$  es justamente un AGM.